Clases y objetos en C++

Mediciones ambientales para El Sereno

Francesco Boito Díaz

Informática II · UTN Facultad Regional Mendoza

Octubre de 2026

Qué resuelve el programa

Cada medición almacena temperatura y humedad relativa. Las mediciones de los nodos agregan un identificador de origen.

Alcance

Los datos son simulados y se muestran en consola. El ejemplo no lee sensores ni transmite por LoRa.

- Actividad 30-09: clase, atributos, métodos, constructor y objetos.
- Actividad 09-09: además, una clase derivada por herencia.

Una clase define el tipo; un objeto tiene los datos

```
Medicion ambiente(18.5f, 55.0f);
```

- `Medicion` es el tipo que definimos.
- `ambiente` es un objeto concreto de ese tipo.
- Cada objeto conserva su propio estado.

En este caso, el constructor recibe 18,5 °C y 55 % de humedad. La letra `f` indica un literal de tipo `float`.

Atributos privados: el estado de cada medición

```
class Medicion {
private:
    float temperatura; // Grados Celsius
    float humedad;     // Humedad relativa en %
    // Aqui continuan los miembros publicos.
};
```

`float` representa valores con decimales. Las unidades forman parte del significado de los datos.

Encapsulamiento

`private` impide acceder directamente a estos atributos desde `main` y desde la clase hija. Se utilizan los métodos públicos.

La definición completa de una clase termina con `};`.

El constructor inicializa el objeto

```
public:
    Medicion(float temperaturaInicial, float humedadInicial)
        : temperatura(temperaturaInicial),
          humedad(humedadInicial) {
    }
```

- Se llama igual que la clase y no declara tipo de retorno.
- Los argumentos se copian a los parámetros.
- Después de `:`, la lista inicializa los atributos.
- El cuerpo está vacío porque la inicialización ya se realizó.

Un método modifica el estado

```
void actualizar(float nuevaTemperatura, float nuevaHumedad) {
    temperatura = nuevaTemperatura;
    humedad = nuevaHumedad;
}
```

`void` indica que no devuelve un valor. Las asignaciones cambian los atributos del objeto que recibe la llamada.

Supuesto del ejemplo

Se ingresan valores válidos de humedad entre 0 y 100 %. Esta versión no implementa validación.

Un método consulta y muestra los datos

```
void mostrar() const {
    std::cout << "Temperatura: " << temperatura << " C\n";
    std::cout << "Humedad: " << humedad << " %\n";
}
```

- `std::cout` escribe en la consola y « envía texto o valores.
- `\n` genera un salto de línea.
- El `const` final indica que el método no modifica el estado del objeto.

La herencia agrega el nodo de origen

```
class MedicionNodo : public Medicion {
private:
    int identificador;
    // Aqui continuan el constructor y el metodo propio.
};
```

`MedicionNodo` es una `Medicion` con un dato adicional: el identificador.

- La herencia pública conserva públicos los métodos públicos de la base.
- `actualizar()` y `mostrar()` quedan disponibles.
- Los atributos privados de la base siguen existiendo, pero la hija no puede acceder a ellos directamente.

El constructor de la hija construye también la base

```
MedicionNodo(int id, float temperaturaInicial,
             float humedadInicial)
    : Medicion(temperaturaInicial, humedadInicial),
      identificador(id) {
}
```

Para `MedicionNodo nodo1(1, 6.5f, 70.0f)`:

1. La base inicializa temperatura en 6,5 y humedad en 70.
2. El atributo propio `identificador` se inicializa en 1.
3. Finalmente se ejecuta el cuerpo vacío.

Aquí declaramos un constructor propio para la hija.

Reutilización del método de la base

```
void mostrarNodo() const {
    std::cout << "Nodo: " << identificador << '\n';
    Medicion::mostrar();
}
```

`mostrarNodo()` imprime el identificador y llama al método de la clase base.

Operador de resolución de ámbito

`Medicion::mostrar()` especifica qué método se utiliza. Evita repetir el código que imprime temperatura y humedad.

Este ejemplo no necesita funciones virtuales ni polimorfismo.

Creación y utilización desde main

```
Medicion ambiente(18.5f, 55.0f);
MedicionNodo nodo1(1, 6.5f, 70.0f);
MedicionNodo nodo2(2, 9.0f, 62.0f);

ambiente.mostrar();
nodo1.mostrarNodo();
nodo1.actualizar(3.0f, 80.0f);
nodo1.mostrarNodo();
nodo2.mostrarNodo();
```

- Las primeras líneas crean e inicializan objetos.
- El punto permite llamar a sus métodos.
- `nodo1.actualizar()` usa un método heredado.
- Los objetos locales se destruyen automáticamente al salir de `main`.

La actualización afecta solamente a nodo1

Objeto	Identificador	Temperatura	Humedad
`ambiente`	No tiene	18,5 °C	55 %
`nodo1`, inicial	1	6,5 °C	70 %
`nodo1`, actualizado	1	3,0 °C	80 %
`nodo2`	2	9,0 °C	62 %

Cada objeto conserva sus propios atributos. La herencia comparte la definición del comportamiento, sin compartir una única temperatura entre los objetos.

Compilación y ejecución en Linux

```
g++ -std=c++11 -Wall -Wextra -pedantic \
    el_sereno.cpp -o el_sereno

./el_sereno
```

1. Abrir una terminal en la carpeta del archivo.
2. Compilar con el comando indicado.
3. Ejecutar el programa con `./el_sereno`.

`-Wall -Wextra -pedantic` habilita advertencias útiles. El código utiliza C++11 y la biblioteca estándar. En Windows el ejecutable puede llamarse `el_sereno.exe`.

Preguntas para defender el código

- **¿Clase y objeto?** La clase define el tipo; cada objeto contiene sus propios datos.
- **¿Para qué sirve el constructor?** Inicializa un objeto al crearlo.
- **¿Por qué atributos privados?** Para controlar el acceso mediante los métodos.
- **¿Qué agrega la herencia?** Permite extender una medición con el identificador del nodo y reutilizar métodos.
- **¿Qué significa const?** El método no modifica el estado del objeto.
- **¿Por qué no hay sensores?** El objetivo de la actividad es demostrar clases y objetos en consola.